

5 Convolutional Layers and 2 Max Pooling Layers Model

In [1]:

```
pip install --upgrade tf_slim
```

Requirement already satisfied: tf_slim in c:\users\holly\appdata\local\programs\python\python313\lib\site-packages (1.1.0)
Requirement already satisfied: absl-py>=0.2.2 in c:\users\holly\appdata\local\programs\python\python313\lib\site-packages (from tf_slim) (2.3.1)
Note: you may need to restart the kernel to use updated packages.

In [2]:

```
#import tensorflow.compat.v1 as tf  
#tf.disable_v2_behavior()
```

```
import tensorflow as tf
```

```
print(tf.__version__)
```

2.20.0

Load Data

In []:

```
import os  
import cv2  
import numpy as np  
import tensorflow as tf
```

```
def load_dataset(folder_path, img_height=64, img_width=64):  
    images_dir = os.path.join(folder_path, "images_resized")  
    labels_file = os.path.join(folder_path, os.path.basename(folder_path) + "_labels.txt")
```

```
    # Load labels
```

```
    with open(labels_file, "r") as f:  
        labels = [int(line.strip()) for line in f]
```

```
    # Get image files in original order
```

```
    image_files = [f for f in os.listdir(images_dir) if f.lower().endswith((".png", ".jpg", ".jpeg"))]
```

```
    if len(image_files) != len(labels):  
        raise ValueError(f"Mismatch: {len(image_files)} images vs {len(labels)} labels")
```

```
    X = []
```

```
    for filename in image_files:  
        img_path = os.path.join(images_dir, filename)  
        img = cv2.imread(img_path)  
        if img is None:  
            raise ValueError(f"Failed to load image: {img_path}")  
        img = cv2.resize(img, (img_width, img_height))  
        X.append(img)
```

```
    X = np.array(X, dtype=np.float32) / 255.0
```

```
    y = np.array(labels, dtype=np.int32)
```

```
    return X, y
```

```
train_path = r"C:\Users\holly\OneDrive\Desktop\642\Research Paper\DRAFT01\archive\train"  
test_path = r"C:\Users\holly\OneDrive\Desktop\642\Research Paper\DRAFT01\archive\test"
```

```
X_train, y_train = load_dataset(train_path)
X_test, y_test = load_dataset(test_path)

X_valid, X_train = X_train[:5000], X_train[5000:]
y_valid, y_train = y_train[:5000], y_train[5000:]
```

In []:

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import numpy as np

def reset_graph(seed=42):
    tf.reset_default_graph()
    tf.set_random_seed(seed)
    np.random.seed(seed)

reset_graph()

height = 64
width = 64
channels = 3
n_outputs = 6

conv_filters = [32, 64, 64, 128, 256]
conv_ksizes = [3]*5
conv_strides = [1]*5
conv_pads = ["SAME"]*5
pool_size = 2
pool_stride = 2
n_fc1 = 128

X = tf.placeholder(tf.float32, shape=[None, height, width, channels], name="X")
y = tf.placeholder(tf.int32, shape=[None], name="y")
training = tf.placeholder_with_default(False, shape=(), name='training')

conv = X
for i in range(5):
    conv = tf.keras.layers.Conv2D(
        filters=conv_filters[i],
        kernel_size=conv_ksizes[i],
        strides=conv_strides[i],
        padding=conv_pads[i],
        activation='relu',
        name=f"conv{i+1}"
    )(conv)

    if i in [1, 3]:
        conv = tf.keras.layers.MaxPooling2D(pool_size=pool_size, strides=pool_stride, padding="VALID")(conv)
        def dropout_layer(x, rate, training):
            return tf.cond(training,
                lambda: tf.nn.dropout(x, rate=rate),
                lambda: x)
        conv = dropout_layer(conv, rate=0.25, training=training)

flat = tf.keras.layers.Flatten()(conv)
fc1 = tf.keras.layers.Dense(n_fc1, activation='relu', name="fc1")(flat)
fc1_drop = dropout_layer(fc1, rate=0.5, training=training)

logits = tf.keras.layers.Dense(n_outputs, name="output")(fc1_drop)
Y_proba = tf.nn.softmax(logits, name="Y_proba")
```

```

xentropy = tf.losses.sparse_softmax_cross_entropy(labels=y, logits=logits)
optimizer = tf.train.AdamOptimizer()
training_op = optimizer.minimize(xentropy)

correct = tf.nn.in_top_k(logits, y, 1)
accuracy = tf.reduce_mean(tf.cast(correct, tf.float32))

init = tf.global_variables_initializer()
saver = tf.train.Saver()

def shuffle_batch(X_data, y_data, batch_size):
    rnd_idx = np.random.permutation(len(X_data))
    for batch_idx in np.array_split(rnd_idx, len(X_data) // batch_size):
        yield X_data[batch_idx], y_data[batch_idx]

n_epochs = 20
batch_size = 10
patience = 20
best_loss = np.inf
checks_without_progress = 0

with tf.Session() as sess:
    sess.run(init)

    for epoch in range(n_epochs):
        for X_batch, y_batch in shuffle_batch(X_train, y_train, batch_size):
            sess.run(training_op, feed_dict={X: X_batch, y: y_batch, training: True})

        loss_val, acc_val = sess.run([xentropy, accuracy],
                                     feed_dict={X: X_valid, y: y_valid})

        if loss_val < best_loss:
            best_loss = loss_val
            checks_without_progress = 0
            saver.save(sess, "./my_fire_model_rgb")
        else:
            checks_without_progress += 1

        print(f"Epoch {epoch+1}, Val Loss: {loss_val:.4f}, Val Acc: {acc_val:.4f}")

        if checks_without_progress > patience:
            print("Early stopping!")
            break

    saver.restore(sess, "./fire_model")
    acc_test = sess.run(accuracy, feed_dict={X: X_test, y: y_test})
    print(f"Test Set Accuracy: {acc_test:.4f}")

```

WARNING:tensorflow:From C:\Users\holly\AppData\Local\Temp\ipykernel_20880\4187393649.py:2: The name tf.disable_v2_behavior is deprecated. Please use tf.compat.v1.disable_v2_behavior instead.

WARNING:tensorflow:From c:\Users\holly\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\python\compat\v2_compat.py:98: disable_resource_variables (from tensorflow.python.ops.resource_variables_toggle) is deprecated and will be removed in a future version.

Instructions for updating:

non-resource variables are not supported in the long term

WARNING:tensorflow:From C:\Users\holly\AppData\Local\Temp\ipykernel_20880\4187393649.py:80: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

WARNING:tensorflow:From C:\Users\holly\AppData\Local\Temp\ipykernel_20880\4187393649.py:81: The name tf.train.AdamOptimizer is deprecated. Please use tf.compat.v1.train.AdamOptimizer instead.

```
Epoch 1, Val Loss: 2.2209, Val Acc: 0.2708
Epoch 2, Val Loss: 1.9620, Val Acc: 0.3568
Epoch 3, Val Loss: 2.5222, Val Acc: 0.4374
Epoch 4, Val Loss: 1.8560, Val Acc: 0.4764
Epoch 5, Val Loss: 2.2068, Val Acc: 0.4680
Epoch 6, Val Loss: 2.4620, Val Acc: 0.5004
Epoch 7, Val Loss: 2.9575, Val Acc: 0.4924
Epoch 8, Val Loss: 2.8992, Val Acc: 0.5372
Epoch 9, Val Loss: 2.5942, Val Acc: 0.5472
Epoch 10, Val Loss: 3.3733, Val Acc: 0.5300
Epoch 11, Val Loss: 3.4796, Val Acc: 0.5966
Epoch 12, Val Loss: 2.9858, Val Acc: 0.5714
Epoch 13, Val Loss: 3.9092, Val Acc: 0.5312
Epoch 14, Val Loss: 3.1948, Val Acc: 0.5756
Epoch 15, Val Loss: 3.2190, Val Acc: 0.6292
Epoch 16, Val Loss: 3.4790, Val Acc: 0.5482
Epoch 17, Val Loss: 3.1021, Val Acc: 0.5850
Epoch 18, Val Loss: 3.9425, Val Acc: 0.5300
Epoch 19, Val Loss: 3.2019, Val Acc: 0.6150
Epoch 20, Val Loss: 3.4324, Val Acc: 0.5952
INFO:tensorflow:Restoring parameters from ./my_fire_model_rgb
□ Final Test Accuracy: 0.9194
```